FACULDADE VINCIT

Organização e Manipulação de Estrutura de Dados

9. Tabela de Dispersão

Tabelas de Dispersão

FACULDADE VINCIT

Tabelas de dispersão, também conhecidas como Tabelas Hashes, ou Hash Tables, são estruturas de indexação de elementos cuja principal finalidade é otimizar a busca de dados em grandes volumes e arranjos.

A história da invenção da tabela de dispersão é incerta. Há duas vertentes, sendo a primeira creditada Hans Peter Luhn em 1953. Luhn foi um pesquisador alemão nos campos da Ciência da Computação, Biblioteconomia e Ciência da Informação para a IBM, sendo mais conhecido por ter sido o criador do algoritmo de Luhn, da indexação KWIC e da divulgação seletiva de informações.

A outra hipótese é de que a Tabela de Dispersão fora inventada pelos autores de compiladores das linguagens de programação da década de 1960, como um truque para manter as listas de variáveis de usuário e seus respectivos valores.

Tabelas de Dispersão - Utilidade

FACULDADE VINCIT

Tabelas Hashes são tipicamente usadas para indexação de grandes volumes de informação. Muitos bancos de dados relacionais disponibilizam índices para serem utilizados em colunas a fim de otimizar a busca de informações. Esses índices são conhecidos como “hash index”.

Alguns interpretadores de linguagens de programação, como o BASIC, utilizam tabelas de dispersão para manter valores de variáveis durante a execução de um programa. Aliás, um interpretador BASIC nada mais é do que uma calculadora com uma tabela de dispersão para controle das variáveis.

Outros exemplos de uso das tabelas de dispersão são as tabelas de transposição em jogos de xadrez para computador até mesmo em serviços de DHCP.

Função de espalhamento

FACULDADE VINCIT

A função de espalhamento, popularmente designada como função hash ou função modular, é um método de gerar um índice a partir de uma chave.

O nome função modular vem do fato de que escolhemos um determinado valor de base para cálculo do índice, sendo, geralmente, uma operação de divisão, em que o resto da divisão da chave pela base retorna um índice (MOD).

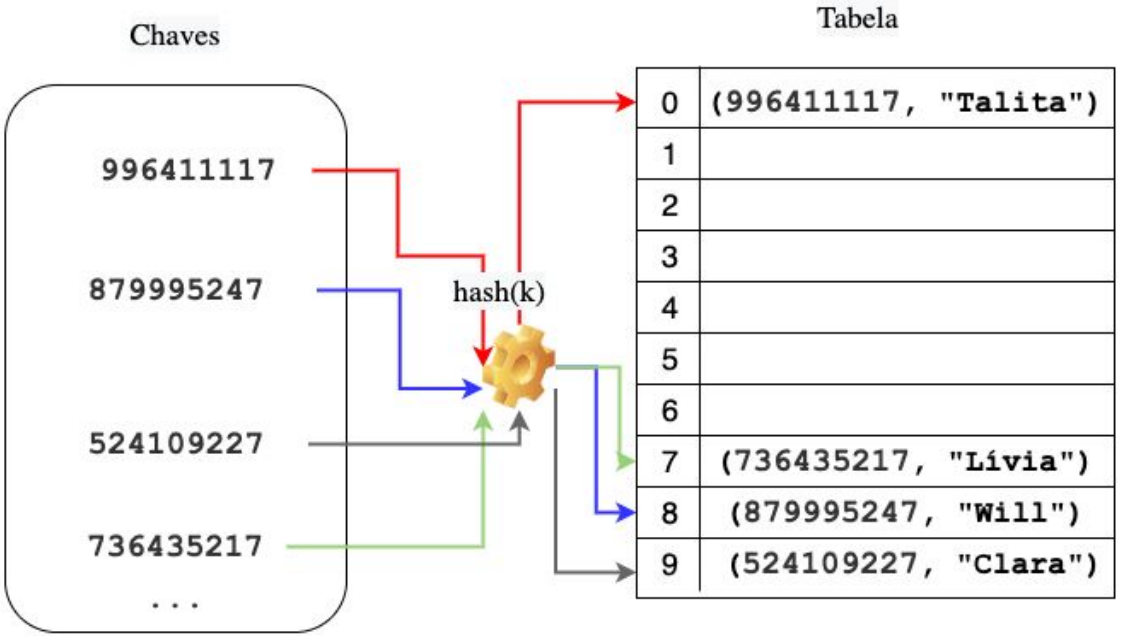
$$f(hash) = key \,\%\, base$$

Ex.:

$$f(hash) = 5512 \,\%\, 10 \rightarrow 2$$

Função de espalhamento

FACULDADE VINCIT



* hash(k) = ((k % tabela.length) + sondagem) % tabela.length
* sondagem pertence à {0...tabela.length - 1}

Tipos de Indexação

FACULDADE VINCIT

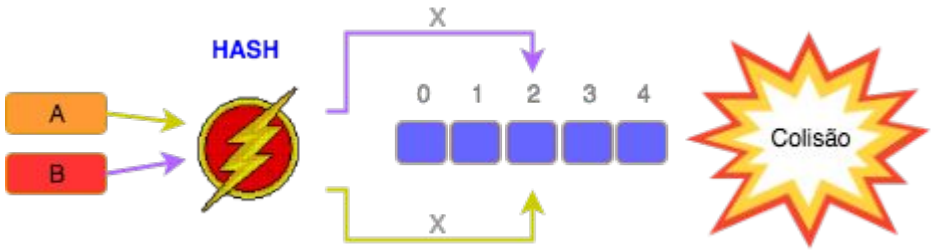
Os dois tipos de indexação mais populares para tabelas hash são:

- **Endereçamento Direto**: Cada chave é mapeada diretamente para uma posição única na tabela usando uma função hash. É uma abordagem simples, porém ineficiente em casos com chaves muito espaçadas.
- **Hashing com Tratamento de Colisões**: Nem sempre é possível evitar colisões (duas ou mais chaves mapeadas em um mesmo índice), porém existem várias técnicas para para lidar com as colisões.
  - ➢ **Encadeamento**: Cada posição na tabela - ou bucket - contém uma lista vinculada, onde todas as chaves mapeadas estão armazenadas nessa lista.
  - ➢ **Endereçamento aberto**: A tabela é percorrida de acordo com uma determinada estratégia (geralmente linear), até encontrar uma posição vazia.

Hashing com Tratamento de Colisões

FACULDADE VINCIT

No Hashing com tratamento de colisões, devemos considerar que em algumas hipóteses, a função hash vai encontrar o mesmo índices para mais de um valor.



Isso é completamente compreensível, pois mesmo a melhor função hash, com a melhor estratégia de cálculo, sempre irá encontrar um índice para mais de um valor.

Se levarmos em consideração uma função de hash modular, mesmo utilizando um número primo como divisor, como é o caso do número 7, em um universo 10 chaves, de 0 a 10, ainda teríamos 3 colisões.

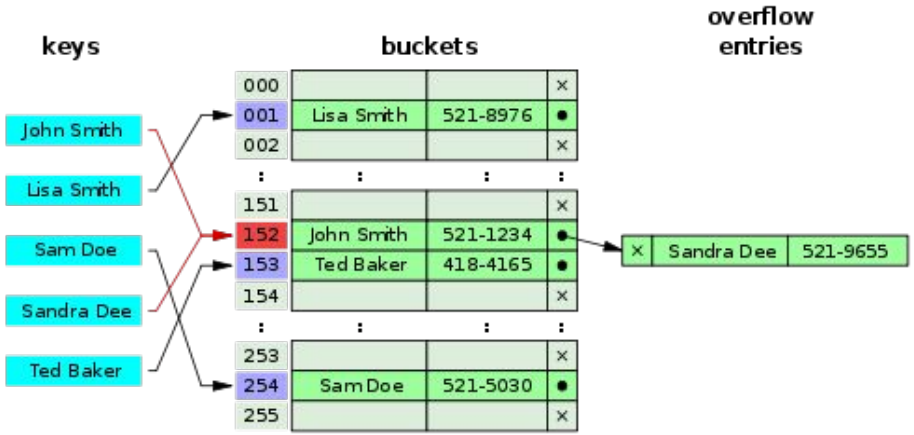
Hashing com Encadeamento

FACULDADE VINCIT

A primeira forma se se resolver as colisões é implementando o conceito de buckets, ou baldes de dados.

Nessa técnica, assumimos que cada índice é um balde para armazenar as chaves encontradas, sendo representadas por uma lista encadeada.

Sempre que encontrarmos uma índice de um valor que já possui um elemento, adicionamos o outro ao final da lista.



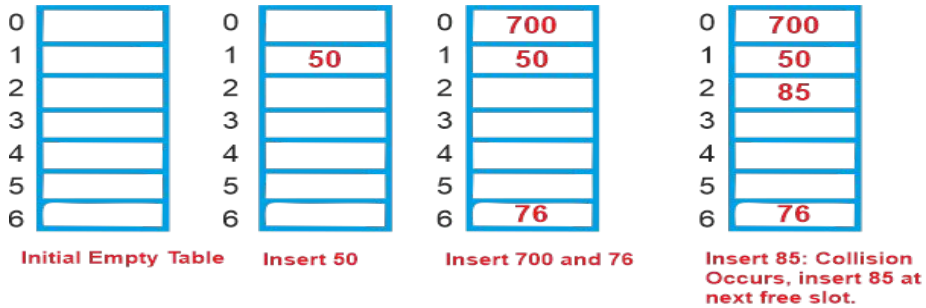
Hashing com Endereçamento Aberto

FACULDADE VINCIT

Outra forma de resolver as colisões é forçando o endereçamento para uma próxima posição válida. Assim, ao encontrar uma colisão de dados, o algoritmo deve encontrar a próxima posição válida e adicionar o elemento nessa próxima posição.

Caso o algoritmo chegue ao final da tabela, o ponteiro de verificação deve ser direcionado ao início da tabela, procurando a próxima posição válida.

O exemplo ao lado considera a fórmula chave MOD 7 como função hash.



FACULDADE VINCIT

fb.com/faculdade.vincit

linkedin.com/school/faculdade_vincit

instagram.com/faculdade_vincit